ParkingHub

Código completo explicado
index.html, style.css e script.js

Equipe

Paulo Henrique Alves de Almeida

Matheus Avanzo dos Santos

Murilo Roselini

Petterson Augusto Papa de Souza

Raphael Souza Araujo Barbosa

Cada trecho de código aparece exatamente como está no projeto, com o número das linhas, seguido da explicação do que ele faz.

Antes do código: como tudo se conecta

O ParkingHub é um site feito só com três arquivos. Cada um tem um papel:

Arquivo	Papel	Analogia
`index.html`	Estrutura: quais elementos existem na página.	O esqueleto e as paredes de uma casa.
`style.css`	Aparência: cores, tamanhos, posições.	A pintura e a decoração.
`script.js`	Comportamento: regras, cálculos, validações.	A parte elétrica: faz as coisas funcionarem.

Como se ligam: o HTML dá um `id` a cada elemento importante (ex.: `id="nome"`). O JavaScript encontra o elemento por esse id (`$("nome")`), lê ou altera seu conteúdo e liga e desliga classes e atributos. O CSS reage a essas classes e atributos (ex.: `.spot.sel` fica laranja).

Ordem recomendada de explicação

Primeiro o HTML (o que existe), depois o CSS (como parece) e por último o JavaScript (como funciona). No JavaScript, siga a ordem do próprio arquivo: ele está organizado como o usuário usa o site.

Glossário rápido

Termo	Significado
Variável (`const` / `let`)	Um nome que guarda um valor. `const` não pode ser reatribuída; `let` pode.
Função	Um bloco de código com nome, que pode ser chamado várias vezes.
Função de seta (`=>`)	Forma curta de escrever função: `x => x * 2`.
Objeto	Um conjunto de propriedades: `{ nome: "Ana", placa: "ABC1D23" }`.
Array	Uma lista de itens: `[1, 2, 3]`.
Template string	Texto entre crases com `${...}` para inserir valores.
Evento	Algo que acontece na página (clique, digitação, envio). `onclick`, `oninput`, `onsubmit` dizem o que fazer quando acontece.
DOM	A página vista pelo JavaScript como uma árvore de elementos que pode ser lida e alterada.
Regex	Padrão que descreve a forma de um texto; serve para validar placa, telefone e nome.
localStorage	Pequeno armazenamento do navegador: guarda texto e sobrevive ao recarregar a página.
JSON	Formato de texto para representar objetos e listas (`JSON.stringify` converte para texto; `JSON.parse` volta para objetos).
Milissegundos	Unidade de tempo que o JavaScript usa para datas: 1 hora = 3.600.000 ms.

Parte 1 - index.html

Cabeçalho do documento (head) (linhas 1 a 13)

```
 1  <!doctype html>
 2  <!-- ParkingHub - estrutura da página (versão simples).
 3       O HTML diz O QUE existe na página. Cada elemento importante tem um "id", que o JavaScript usa para
achá-lo.
 4       O atributo "class" serve para o CSS dar o visual. O atributo "hidden" esconde o elemento até o
JavaScript mostrá-lo. -->
 5  <html lang="pt-BR">
 6  <head>
 7    <meta charset="UTF-8">                                          <!-- permite acentos e
caracteres especiais -->
 8    <meta name="viewport" content="width=device-width, initial-scale=1.0">  <!-- ajusta a página à largura
do celular -->
 9    <title>ParkingHub · Reserva de vagas</title>                    <!-- texto da aba do navegador
-->
10    <link rel="icon" href="image/logo-provisoria.jpeg">                <!-- ícone da aba -->
11    <link rel="stylesheet" href="style.css">                        <!-- liga o arquivo de
aparência -->
12    <script src="script.js" defer></script>                         <!-- liga a lógica; "defer"
espera a página carregar antes de rodar -->
13  </head>
```

- `<!doctype html>` avisa ao navegador que é HTML moderno (HTML5).
- `lang="pt-BR"` informa o idioma da página (ajuda leitores de tela e corretor ortográfico).
- `<meta charset="UTF-8">` permite acentos e caracteres especiais.
- `<meta name="viewport">` faz a página se ajustar à largura do celular.
- `<title>` é o texto da aba do navegador.
- `<link rel="icon">` define o ícone da aba. `<link rel="stylesheet">` liga o arquivo `style.css`.
- `<script src="script.js" defer>` liga o JavaScript. O `defer` faz o navegador só executá-lo **depois** de ler a página inteira, para que os elementos já existam quando o JS for procurá-los.

Filtro da logo e cabeçalho da página (linhas 14 a 36)

```
14  <body>
15    <!-- Filtro invisível: no tema claro, troca o preto da logo por branco (o CSS usa o id "logo-claro") -
->
16    <svg width="0" height="0" style="position:absolute"><filter id="logo-claro" color-interpolation-
filters="sRGB"><feColorMatrix type="matrix" values="1 0 -1 0 0  0 1 -1 0 0  0 0 0 0 0  1 0 0 0 0"/></filter>
</svg>
17
18    <!-- Biblioteca de ícones (invisível): cada <symbol> é um desenho que o resto da página reutiliza com
<use href="#icone-...">.
19         Os ícones usam a cor do texto onde forem colocados. -->
20    <svg width="0" height="0" style="position:absolute" aria-hidden="true">
21      <symbol id="icone-lua" viewBox="0 0 24 24"><path d="M20.8 13.1A9 9 0 0 1 10.9 3.2 9 9 0 1 0 20.8
13.1Z"/></symbol>
22      <symbol id="icone-sol" viewBox="0 0 24 24"><circle cx="12" cy="12" r="4"/><path d="M12 2v2m0 16v2M2
12h2m16 0h2M5 5l1.5 1.5m11 11L19 19M5 19l1.5-1.5m11-11L19 5"/></symbol>
23      <symbol id="icone-moto" viewBox="0 0 24 24"><circle cx="5" cy="17" r="3.5"/><circle cx="19" cy="17"
r="3.5"/><path d="M19 17 15 6h-3m-9 4h5l3 4h5M5 17l4-5m1 2 1 3h4m-7-7 4-2h4"/><path d="M8 10h4l3 3H10Z"
fill="currentColor" stroke="none"/></symbol>
24      <symbol id="icone-acessivel" viewBox="0 0 24 24"><circle cx="11" cy="4" r="1.5"/><path d="M11
7v6h6l3 7m-9-10h6M8 11a6 6 0 1 0 8 8"/></symbol>
```

```
  </svg>

  <!-- CABEÇALHO: logo, nome do sistema e botão que alterna o tema claro/escuro -->
  <header class="topo">
    <div class="miolo">
      <a class="marca" href="">
        <span class="logo"><img src="image/logo-provisoria.jpeg" alt="Logo"></span>
        <span>ParkingHub<small>Reserva de vagas</small></span>
      </a>
      <button class="icone" id="tema" type="button" aria-label="Alternar tema"><svg class="icone-svg" aria-hidden="true"><use href="#icone-lua"/></svg></button> <!-- o ícone (lua ou sol) é trocado pelo JavaScript -->
    </div>
  </header>
```

- O `<svg>` com `<filter>` é um truque visual: no tema claro, ele troca o preto da logo por branco para ela aparecer bem. Está invisível (largura 0). O CSS chama o filtro pelo id `logo-claro`.
- O segundo `<svg>` é a **biblioteca de ícones**: cada `<symbol>` (lua, sol, moto e acessível) é um desenho com um id. Em qualquer lugar, `<use href="#icone-moto">` reaproveita o desenho. Assim não precisamos de emojis nem de arquivos de imagem.
- `<header class="topo">` é o cabeçalho. Dentro dele, a logo, o nome "ParkingHub" e o botão `#tema` (ícone de lua ou sol) que o JS usa para alternar o tema.
- `class` serve para o CSS estilizar; `id` é um nome único para o JavaScript achar o elemento.

Título e barra de etapas (linhas 38 a 43)

```
  <main class="miolo pagina">
    <h1>Reserve sua vaga</h1>
    <p class="sub">Escolha sua vaga, informe seus dados e siga para o pagamento.</p>

    <!-- Barra de progresso: o JavaScript marca qual etapa está ativa -->
    <ol class="etapas" id="etapas"><li>Vaga e período</li><li>Seus dados</li><li>Pagamento</li></ol>
```

- `<main>` é o conteúdo principal da página.
- `<ol id="etapas">` é uma lista numerada com as 3 etapas. O JS troca a classe de cada `<li>` para destacar a etapa atual (`atual`) e as já feitas (`feito`).

Etapa 1: período da reserva (linhas 45 a 63)

```
    <!-- ===== ETAPA 1: período, planta e resumo ===== -->
    <form class="reserva" id="tela-vaga" novalidate>

      <!-- Período: modalidade, data e horários -->
      <section class="periodo">
        <h2>Período da reserva</h2>
        <fieldset class="modo">
          <legend>Modalidade</legend>
          <!-- Os dois botões de rádio têm o mesmo "name", então só um pode ficar marcado -->
          <label><input type="radio" name="modalidade" value="horario" checked> Por horário</label>
          <label><input type="radio" name="modalidade" value="diaria"> Diária de 24 h</label>
        </fieldset>
        <label class="campo">Data de entrada <input type="date" id="data"></label>
        <div class="duas">
          <label class="campo">Entrada <input type="time" id="entrada" value="08:00"></label>
          <label class="campo" id="campo-saida">Saída <input type="time" id="saida" value="10:00">
</label> <!-- some na diária -->
        </div>
```

```
62          <p class="erro" id="erro-periodo" hidden></p> <!-- mensagem de período inválido -->
63        </section>
```

- `<form id="tela-vaga">` agrupa toda a etapa 1. O JS captura o envio (clique em "Continuar") deste formulário.
- Dois `<input type="radio">` com o mesmo `name="modalidade"` formam um grupo em que só um pode ser marcado: "horario" (por horário) ou "diaria". O `value` é o que o JS lê.
- `type="date"` e `type="time"` mostram o seletor de data e de hora do próprio navegador. Os valores iniciais de horário (08:00 e 10:00) evitam que a tela comece sem período válido.
- `#campo-saida` é o campo de saída; o JS o esconde quando a modalidade é diária.
- `<p id="erro-periodo" hidden>` mostra mensagens de erro do período. O atributo `hidden` mantém o parágrafo oculto até ser necessário.

Etapa 1: planta do estacionamento (linhas 65 a 87)

```
65        <!-- Planta: as fileiras estão VAZIAS; o JavaScript cria as 34 vagas dentro delas -->
66        <section class="planta-box">
67          <div class="titulo-planta">
68            <div><h2>Escolha uma vaga</h2><p>Planta do estacionamento</p></div>
69            <span id="disponiveis"></span> <!-- "34 de 34 disponíveis" -->
70          </div>
71          <div class="legenda">
72            <span><i class="livre"></i>Disponível</span>
73            <span><i class="selecionada"></i>Selecionada</span>
74            <span><i class="reservada"></i>Reservada</span>
75            <span><svg class="icone-svg" aria-hidden="true"><use href="#icone-acessivel"/></svg>
Acessível</span>
76          </div>
77          <div class="rolagem">
78            <div class="planta">
79              <div class="muro"></div>
80              <div class="fileira topo-f" id="fileira-topo"></div>
81              <div class="fileira meio-f" id="fileira-meio"></div>
82              <div class="fileira base-f" id="fileira-base"></div>
83              <div class="entrada"><span>←<small>Saída</small></span><span>→<small>Entrada</small></span>
</div>
84            </div>
85          </div>
86          <p class="legenda-fim">V1–V26: carros · M1–M6: motos · D1–D2: acessíveis</p>
87        </section>
```

- O título mostra "N de 34 disponíveis" em `#disponiveis` (preenchido pelo JS).
- A legenda usa tags `<i>` vazias, coloridas pelo CSS, como amostras de cor.
- `.rolagem` permite rolar a planta para o lado em telas pequenas.
- Dentro de `.planta` estão o muro desenhado e as **três fileiras vazias** (`#fileira-topo`, `#fileira-meio`, `#fileira-base`). Os botões das 34 vagas **não estão escritos aqui**: o JavaScript cria cada um dentro dessas divs.
- A div `.entrada` desenha as setas de entrada e saída.

Etapa 1: resumo e botão continuar (linhas 89 a 103)

```
89        <!-- Resumo: vaga, datas, duração e total (preenchidos pelo JavaScript) -->
90        <section class="resumo-box">
91          <h2>Confira sua reserva</h2>
```

```
 92          <div class="vaga-escolhida"><span>Vaga selecionada</span><strong id="r-vaga"></strong><small
id="r-tipo"></small></div>
 93          <dl>
 94            <div><dt>Entrada</dt><dd id="r-entrada"></dd></div>
 95            <div><dt>Saída</dt><dd id="r-saida"></dd></div>
 96            <div><dt>Duração</dt><dd id="r-duracao"></dd></div>
 97            <div class="total"><dt>Total da reserva</dt><dd id="r-total"></dd></div>
 98          </dl>
 99          <button class="principal" id="continuar" disabled>Selecione uma vaga</button> <!-- liberado só
com período e vaga válidos -->
100          <p class="aviso" id="aviso" role="status" hidden></p>                          <!-- mensagens de
sucesso ou de vaga ocupada -->
101          <p class="nota" id="tarifas"></p>
102        </section>
103      </form>
```

- Mostra a vaga selecionada, entrada, saída, duração e total. Cada valor está num `<dd>` com id (`r-vaga` , `r-total` ...) que o JS preenche.
- `<dl>` / `<dt>` / `<dd>` é a lista de definições: nome do dado e seu valor.
- O botão `#continuar` começa `disabled` (desabilitado) e só é liberado quando há período válido e vaga escolhida.
- `#aviso` mostra mensagens (sucesso da reserva, vaga que ficou ocupada) e `#tarifas` mostra os preços.

Etapas 2 e 3: tela de confirmação e resumo (linhas 105 a 121)

```
105      <!-- ===== ETAPAS 2 e 3: o resumo à esquerda é o mesmo; à direita aparece o formulário de dados OU o
de pagamento ===== -->
106      <section class="confirmar" id="tela-confirmar" hidden>
107        <button class="voltar" id="voltar" type="button">← Voltar</button>
108        <div class="duas-colunas">
109
110          <!-- Resumo lateral da reserva -->
111          <aside class="resumo-box">
112            <h2>Sua reserva</h2>
113            <div class="vaga-escolhida"><strong id="c-vaga"></strong><small id="c-tipo"></small></div>
114            <dl>
115              <div><dt>Modalidade</dt><dd id="c-mod"></dd></div>
116              <div><dt>Entrada</dt><dd id="c-entrada"></dd></div>
117              <div><dt>Saída</dt><dd id="c-saida"></dd></div>
118              <div id="c-cliente" hidden><dt>Cliente</dt><dd id="c-nome"></dd></div> <!-- só aparece na
etapa 3 -->
119              <div class="total"><dt id="c-rotulo">Total da reserva</dt><dd id="c-total"></dd></div>
120            </dl>
121          </aside>
```

- `#tela-confirmar` começa `hidden` . Ela é compartilhada pelas etapas 2 e 3: o resumo à esquerda é o mesmo e muda só o formulário à direita.
- `<aside>` é o resumo lateral (vaga, modalidade, entrada, saída, cliente, total). A linha do cliente (`#c-cliente`) só aparece na etapa 3.
- O botão "Voltar" tem `type="button"` para não enviar o formulário.

Etapa 2: formulário de dados (linhas 123 a 137)

```
123          <!-- ETAPA 2: dados do cliente e do veículo. "novalidate": a validação é feita pelo nosso
JavaScript -->
124          <form class="form-box" id="form-dados" novalidate>
```

```
125            <h2>Seus dados</h2>
126            <label class="campo">Nome completo <input id="nome" maxlength="120" autocomplete="name">
</label>
127            <label class="campo">Telefone com DDD <input id="telefone" type="tel" maxlength="19"
placeholder="(11) 99999-9999"></label>
128            <h2>Dados do veículo</h2>
129            <p class="nota" id="ajuda-tipo"></p>
130            <label class="campo">Placa <input id="placa" maxlength="9" placeholder="ABC1D23"
autocomplete="off"></label>
131            <!-- Credencial: só aparece quando a vaga escolhida é acessível (D1 ou D2) -->
132            <label class="campo" id="campo-cred" hidden>Credencial da vaga acessível (JPG, PNG, WebP ou
PDF, até 5 MB)
133              <input type="file" id="credencial" accept="image/jpeg,image/png,image/webp,application/pdf">
134            </label>
135            <p class="erro" id="erro-dados" hidden></p>
136            <button class="principal">Ir para pagamento</button>
137          </form>
```

- Campos de nome, telefone e placa. `maxlength` limita a quantidade de caracteres; `placeholder` mostra o exemplo cinza dentro do campo; `type="tel"` abre o teclado numérico no celular.
- `#campo-cred` (credencial) começa oculto e só aparece em vagas acessíveis. `type="file"` com `accept` filtra os tipos de arquivo no seletor.
- `#erro-dados` exibe o erro de validação.
- `novalidate` desliga a validação automática do navegador, porque a validação é feita pelo nosso JavaScript, com mensagens em português.

Etapa 3: formulário de pagamento (linhas 139 a 155)

```
139          <!-- ETAPA 3: forma de pagamento. O "value" de cada opção é a chave do objeto PAGAMENTOS no
JavaScript -->
140          <form class="form-box" id="form-pagamento" novalidate hidden>
141            <fieldset class="pagamentos">
142              <legend>Forma de pagamento</legend>
143              <label><input type="radio" name="pagamento" value="cartao"> <b>Cartão de crédito</b>
<small>Pagar no estacionamento</small></label>
144              <label><input type="radio" name="pagamento" value="google-wallet"> <b>Google Wallet</b>
<small>Carteira digital</small></label>
145              <label><input type="radio" name="pagamento" value="pix"> <b>Pix</b> <small>Simular com QR
Code</small></label>
146              <label><input type="radio" name="pagamento" value="dinheiro"> <b>Dinheiro</b> <small>Pagar
no estacionamento</small></label>
147            </fieldset>
148            <h2 id="pg-titulo">Escolha uma opção acima</h2>
149            <p id="pg-texto">O total é o mesmo em todas as formas de pagamento.</p>
150            <img id="pix" src="image/pix-demonstracao.svg" width="180" height="180" alt="QR Code
ilustrativo" hidden> <!-- só no Pix -->
151            <p class="nota">Ambiente de demonstração: nenhuma cobrança real é realizada.</p>
152            <button class="principal" id="pagar" disabled>Confirmar reserva</button>
153          </form>
154        </div>
155      </section>
```

- Quatro `radio` com `name="pagamento"`: cartão, Google Wallet, Pix e dinheiro. O `value` de cada um é a chave usada no objeto `PAGAMENTOS` do JS.
- `<fieldset>` e `<legend>` agrupam e nomeiam as opções.
- O título e o texto (`#pg-titulo`, `#pg-texto`) mudam conforme a opção. A imagem do QR Code (`#pix`) só aparece no Pix.

- O botão `#pagar` começa desabilitado até escolher uma forma. Este formulário começa `hidden`.

Minhas reservas (linhas 157 a 165)

```
157     <!-- ===== MINHAS RESERVAS: a lista é desenhada pelo JavaScript dentro de #lista ===== -->
158     <section class="minhas" id="minhas">
159       <div class="titulo-minhas">
160         <h2>Minhas reservas <span id="contador">0</span></h2> <!-- quantidade de reservas ativas -->
161         <button class="botao" id="limpar" type="button">Limpar reservas</button>
162       </div>
163       <div id="lista"></div>
164     </section>
165   </main>
```

- Título com o contador de reservas ativas (`#contador`) e o botão "Limpar reservas".
- `<div id="lista">` começa vazia. O JS desenha ali um cartão para cada reserva.

Rodapé e janela de saída (linhas 167 a 194)

```
167   <!-- RODAPÉ: <details> abre e fecha ao clicar, sem precisar de JavaScript -->
168   <footer class="rodape miolo">
169     <span>Projeto de gerenciamento de estacionamento</span>
170     <details><summary>Equipe do projeto</summary>
171       <ul>
172         <li>Paulo Henrique Alves de Almeida</li>
173         <li>Matheus Avanzo dos Santos</li>
174         <li>Murilo Roselini</li>
175         <li>Petterson Augusto Papa de Souza</li>
176         <li>Raphael Souza Araujo Barbosa</li>
177       </ul>
178     </details>
179   </footer>
180
181   <!-- JANELA DE SAÍDA: <dialog> é uma janela nativa; o JavaScript a abre com showModal().
182        method="dialog" faz a janela fechar sozinha ao enviar o formulário. -->
183   <dialog id="modal-saida">
184     <h2 id="titulo-saida">Simular saída</h2>
185     <p id="texto-saida"></p>
186     <form id="form-saida" method="dialog">
187       <label class="campo">Data e hora de saída <input type="datetime-local" id="saida-real"></label>
188       <p class="erro" id="erro-saida" hidden></p>
189       <div id="resultado-saida"></div> <!-- cálculo da taxa de atraso -->
190       <div class="acoes"><button class="botao" type="button" id="fechar-saida">Fechar</button><button
class="principal">Encerrar reserva</button></div>
191     </form>
192   </dialog>
193 </body>
194 </html>
```

- `<footer>` traz a equipe dentro de `<details>`, um bloco que abre e fecha ao clicar em `<summary>`, sem precisar de JavaScript.
- `<dialog>` é uma janela modal nativa. O JS a abre com `showModal()` e fecha com `close()`.
- O formulário da janela usa `method="dialog"`: ao enviar, o navegador fecha a janela sozinho. Dentro dela: campo de data e hora de saída (`datetime-local`), mensagem de erro e a área `#resultado-saida` com o cálculo.

Parte 2 - style.css

Fontes (linhas 1 a 6)

```
1  /* ParkingHub - aparência do sistema (versão simples).
2     O CSS só cuida do visual: o JavaScript liga e desliga classes e atributos, e as regras abaixo reagem
a eles. */
3
4  /* Fontes locais (pasta fonts): Source Sans 3 para o texto e Barlow para os títulos. */
5  @font-face { font-family: "Source Sans 3"; src: url("fonts/SourceSans3-Variable.ttf"); font-weight: 200
900; }
6  @font-face { font-family: "Barlow"; src: url("fonts/Barlow-SemiBold.ttf"); font-weight: 600; }
```

- `@font-face` carrega as fontes da pasta `fonts`: Source Sans 3 (texto) e Barlow (títulos). As fontes ficam na pasta `fonts`, na raiz do projeto.

Variáveis de cor e tema escuro (linhas 8 a 39)

```
8  /* Cores do tema claro, guardadas em variáveis (usadas com var(--nome)).
9     Mudar uma cor aqui muda o site inteiro. */
10 :root {
11   --fundo: #f5f6f7;            /* fundo da página */
12   --superficie: #fff;          /* fundo de cartões, campos e botões */
13   --suave: #e9ecef;            /* cinza claro de apoio */
14   --texto: #191d21;            /* cor do texto principal */
15   --secundario: #505c66;       /* cor de textos de apoio */
16   --borda: #cdd3d8;            /* cor das bordas */
17   --destaque: #c44100;         /* laranja da marca */
18   --destaque-suave: #fff1e7;   /* laranja bem claro (fundos de destaque) */
19   --sobre-destaque: #fff;      /* cor do texto sobre o laranja */
20   --piso: #e9ecef;             /* piso do estacionamento */
21   --marcacao: #77838e;         /* linhas desenhadas na planta */
22   --reservada: #d6dadd;        /* cinza das vagas reservadas */
23   --azul: #24669a;             /* azul das vagas acessíveis */
24   --vermelho: #ab3522;         /* mensagens de erro */
25   --cabecalho: #e9eaec;        /* fundo do cabeçalho */
26   --cabecalho-texto: #17191c;  /* texto do cabeçalho */
27   --fonte-texto: "Source Sans 3", "Segoe UI", sans-serif;
28   --fonte-titulo: "Barlow", "Segoe UI", sans-serif;
29 }
30
31 /* Tema escuro: só REDEFINE as mesmas variáveis. Vale quando o elemento raiz tem data-tema="escuro"
32    (o JavaScript escreve esse atributo no botão de tema). */
33 :root[data-tema="escuro"] {
34   color-scheme: dark;
35   --fundo: #101214; --superficie: #16191c; --suave: #242a30; --texto: #f0f1f2; --secundario: #b0bac3;
36   --borda: #343b42; --destaque: #ff7b29; --destaque-suave: #35251b; --sobre-destaque: #101214;
37   --piso: #0c0f11; --reservada: #323940; --azul: #88bce8; --vermelho: #ffb0a0;
38   --cabecalho: #000; --cabecalho-texto: #fff;
39 }
```

- `:root` é o elemento raiz da página. As variáveis `--nome` guardam as cores e são usadas com `var(--nome)`. Mudar uma cor aqui muda o site todo.
- `:root[data-tema="escuro"]` só vale quando o elemento raiz tem `data-tema="escuro"`. Ele **redefine as mesmas variáveis** com cores escuras. É assim que o tema escuro funciona: o JS troca o atributo e o CSS faz o resto.
- `--destaque` é o laranja da marca. `--fonte-texto` e `--fonte-titulo` são as fontes.

Estilos base (linhas 41 a 53)

```
41  /* ---------- Estilos base ---------- */
42  * { box-sizing: border-box; }  /* largura e altura incluem borda e espaçamento interno */
43  body { margin: 0; background: var(--fundo); color: var(--texto); font: 16px/1.5 var(--fonte-texto); }
44  h1, h2, p, dl { margin: 0; }
45  h1, h2 { font-family: var(--fonte-titulo); font-weight: 600; }
46  h1 { font-size: 36px; line-height: 1.15; }
47  h2 { font-size: 22px; }
48  button, input, select { font: inherit; }  /* campos usam a mesma fonte da página */
49  button { cursor: pointer; }
50  [hidden] { display: none !important; }    /* qualquer elemento com o atributo "hidden" some de verdade
*/
51  .miolo { width: min(1440px, calc(100% - 64px)); margin: 0 auto; }  /* limita a largura e centraliza o
conteúdo */
52  .pagina { padding: 28px 0 32px; }
53  .sub { color: var(--secundario); font-size: 15px; margin: 4px 0 20px; }
```

- `box-sizing: border-box`: largura e altura incluem borda e espaçamento interno (fica mais fácil calcular tamanhos).
- `body` define cor de fundo, cor do texto e fonte padrão. `[hidden] { display: none !important }` garante que qualquer elemento com o atributo `hidden` realmente suma, mesmo que outra regra tente exibi-lo.
- `.miolo` limita a largura do conteúdo (máximo de 1440 px) e o centraliza com `margin: 0 auto`.

Cabeçalho e logo (linhas 55 a 69)

```
55  /* ---------- Cabeçalho ---------- */
56  .topo { background: var(--cabecalho); color: var(--cabecalho-texto); border-bottom: 1px solid var(--
borda); padding: 8px 0; }
57  .topo .miolo { display: flex; justify-content: space-between; align-items: center; }  /* marca à
esquerda, tema à direita */
58  .marca { display: flex; align-items: center; gap: 12px; color: inherit; text-decoration: none; font: 600
21px/1.3 var(--fonte-titulo); }
59  .marca small { display: block; font: 12px var(--fonte-texto); opacity: .7; }
60  /* A logo é uma imagem maior dentro de uma caixa de 60 px que recorta só o símbolo. */
61  .logo { width: 60px; height: 60px; overflow: hidden; position: relative; }
62  .logo img { position: absolute; width: 112px; left: -26px; top: -24px; filter: url("#logo-claro"); }  /*
filtro do SVG no HTML (tema claro) */
63  :root[data-tema="escuro"] .logo img { filter: none; }
64  .icone { width: 44px; height: 44px; border: 0; background: none; color: inherit; display: inline-flex;
align-items: center; justify-content: center; }
65  /* Ícones SVG: desenhados só com traço, que usa a cor do texto (currentColor). Os desenhos ficam no
index.html. */
66  .icone-svg { width: 19px; height: 19px; flex: 0 0 19px; fill: none; stroke: currentColor; stroke-width:
1.7; stroke-linecap: round; stroke-linejoin: round; }
67  .icone .icone-svg { width: 24px; height: 24px; }                                                    /*
ícone do tema, no cabeçalho */
68  .vaga.acessivel .icone-svg { width: 25px; height: 25px; flex-basis: 25px; }                         /*
vaga acessível: ícone maior, na cor azul da vaga */
69  .legenda .icone-svg { width: 17px; height: 17px; flex-basis: 17px; color: var(--azul); vertical-align:
-4px; }  /* legenda */
```

- `display: flex` com `justify-content: space-between` coloca a marca à esquerda e o botão de tema à direita.
- A logo é uma imagem maior dentro de uma caixa de 60 px com `overflow: hidden`: a caixa recorta só o símbolo. `filter: url("#logo-claro")` aplica o filtro do SVG (só no tema claro).

- `.icone-svg` define o tamanho dos ícones e usa `stroke: currentColor`: o traço do desenho assume a cor do texto ao redor (branco, azul na vaga acessível, etc.). `fill: none` deixa o desenho só com contorno.

Barra de etapas (linhas 71 a 76)

```
 71  /* ---------- Barra de etapas (os números 1., 2., 3. são gerados automaticamente) ---------- */
 72  .etapas { display: flex; gap: 8px; padding: 0; margin: 0 0 24px; list-style: none; counter-reset: etapa;
}
 73  .etapas li { counter-increment: etapa; flex: 1; padding: 8px 0; border-top: 3px solid var(--borda);
color: var(--secundario); font-size: 14px; }
 74  .etapas li::before { content: counter(etapa) ". "; }
 75  .etapas .atual { border-color: var(--destaque); color: var(--texto); font-weight: 600; }  /* etapa em
andamento */
 76  .etapas .feito { border-color: var(--texto); }                                            /* etapas já
concluídas */
```

- Cada `<li>` vira uma faixa com borda superior. `counter-reset` e `counter-increment` numeram automaticamente (1., 2., 3.) pelo `::before`.
- `.atual` (etapa em andamento) fica com borda laranja e negrito; `.feito` fica com borda escura. As classes são colocadas pelo JS.

Campos, botões e mensagens (linhas 78 a 93)

```
 78  /* ---------- Campos, botões e mensagens ---------- */
 79  .campo { display: flex; flex-direction: column; gap: 4px; margin-top: 14px; font-size: 14px; }  /*
rótulo em cima, campo embaixo */
 80  input, select { padding: 9px 10px; border: 1px solid var(--secundario); border-radius: 3px; background:
var(--superficie); color: inherit; }
 81  fieldset { border: 0; padding: 0; margin: 14px 0 0; }
 82  legend { font-size: 14px; padding: 0; margin-bottom: 4px; }
 83  .modo label { margin-right: 16px; }
 84  .duas { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; }  /* dois campos lado a lado */
 85  .principal { width: 100%; margin-top: 16px; padding: 12px; border: 0; border-radius: 3px; background:
var(--destaque); color: var(--sobre-destaque); font-weight: 600; }
 86  .principal:hover:enabled { filter: brightness(1.1); }                              /* clareia ao passar
o mouse */
 87  .principal:disabled { background: var(--suave); color: var(--secundario); cursor: not-allowed; }  /*
botão desabilitado fica cinza */
 88  .botao { padding: 8px 14px; border: 1px solid var(--borda); border-radius: 3px; background: var(--
superficie); color: inherit; }
 89  .botao:hover { background: var(--suave); }
 90  .voltar { border: 0; background: none; color: var(--texto); padding: 8px 0; margin-bottom: 12px; }
 91  .erro { color: var(--vermelho); font-size: 14px; margin-top: 10px; }
 92  .aviso { margin-top: 12px; padding: 10px; border-left: 4px solid var(--destaque); background: var(--
destaque-suave); font-size: 14px; }
 93  .nota { color: var(--secundario); font-size: 13px; margin-top: 8px; }
```

- `.campo`: rótulo e input empilhados (`flex-direction: column`) com espaço entre eles.
- `.principal`: botão laranja de largura total. `:hover` clareia ao passar o mouse e `:disabled` deixa cinza quando está desabilitado.
- `.botao` é o botão secundário (borda cinza). `.erro` é texto vermelho e `.aviso` é a caixa de mensagem com barra laranja à esquerda.

Layout da etapa 1 (linhas 95 a 108)

```
 95  /* ---------- Etapa 1: planta à esquerda, período e resumo à direita (CSS Grid) ---------- */
 96  .reserva { display: grid; grid-template-columns: minmax(0, 1fr) 344px; grid-template-areas: "planta
periodo" "planta resumo"; align-items: start; column-gap: 32px; }
 97  .planta-box { grid-area: planta; background: var(--superficie); border: 1px solid var(--borda);
overflow: hidden; }
 98  .periodo { grid-area: periodo; padding-bottom: 24px; }
 99  .resumo-box { grid-area: resumo; padding-top: 24px; border-top: 1px solid var(--borda); }
100  .titulo-planta { display: flex; justify-content: space-between; align-items: center; padding: 22px 24px
12px; }
101  .titulo-planta p, .titulo-planta span { color: var(--secundario); font-size: 14px; }
102  /* Legenda: quadradinhos coloridos que explicam as cores das vagas. */
103  .legenda { display: flex; flex-wrap: wrap; gap: 18px; padding: 0 24px 16px; color: var(--secundario);
font-size: 13px; }
104  .legenda i { display: inline-block; width: 14px; height: 14px; margin-right: 6px; vertical-align: -2px;
border: 1px solid var(--marcacao); }
105  .legenda .livre { background: var(--superficie); }
106  .legenda .selecionada { background: var(--destaque); }
107  .legenda .reservada { background: var(--reservada); }
108  .legenda-fim { padding: 14px 24px; color: var(--secundario); font-size: 12px; }
```

- `.reserva` usa **CSS Grid**: duas colunas (a primeira ocupa o espaço restante, a segunda tem 344 px). `grid-template-areas` posiciona: a planta ocupa as duas linhas da coluna da esquerda, e período e resumo ficam empilhados na direita.
- `grid-area` em cada bloco diz em qual área ele fica.
- A legenda usa quadradinhos coloridos (`.legenda i`) para mostrar disponível, selecionada e reservada.

A planta e as vagas (linhas 110 a 129)

```
110  /* ---------- Planta: posições em porcentagem, para o desenho acompanhar o tamanho da tela ---------- */
111  .rolagem { overflow-x: auto; border-block: 1px solid var(--borda); }  /* em telas pequenas, rola para o
lado */
112  .planta { position: relative; min-width: 950px; aspect-ratio: 1.286; background: var(--piso); }
113  .muro { position: absolute; inset: 3%; border: 2px solid var(--marcacao); }  /* contorno do
estacionamento */
114  .fileira { position: absolute; display: grid; }  /* cada fileira é uma grade de vagas */
115  .topo-f { left: 19%; top: 7%; width: 69%; height: 18%; grid-template-columns: repeat(7, 1fr); }   /* 7
colunas: motos + V1 a V6 */
116  .meio-f { left: 16%; top: 33%; width: 78%; height: 36%; grid-template-columns: repeat(8, 1fr); grid-
template-rows: 1fr 1fr; }  /* 8 colunas x 2 linhas: 16 vagas */
117  .base-f { left: 19%; top: 77%; width: 69%; height: 18%; grid-template-columns: repeat(7, 1fr); }  /* 7
colunas: motos + V21 a V26 */
118  .motos { display: grid; grid-template-rows: repeat(3, 1fr); }  /* pilha de 3 vagas de moto */
119
120  /* Cada vaga é um botão. */
121  .vaga { display: flex; flex-direction: column; align-items: center; justify-content: center; gap: 4px;
min-width: 0; padding: 4px; border: 1px solid var(--marcacao); background: var(--superficie); color: var(--
texto); font-weight: 600; }
122  .vaga.moto { flex-direction: row; }
123  .vaga.acessivel { color: var(--azul); }
124  .vaga:enabled:hover { background: var(--destaque-suave); outline: 2px solid var(--destaque); outline-
offset: -2px; }  /* destaque ao passar o mouse */
125  .vaga.selecionada { background: var(--destaque); color: var(--sobre-destaque); }
/* vaga escolhida: laranja */
126  .vaga:disabled { background: var(--reservada); color: var(--secundario); cursor: not-allowed; }
/* reservada ou sem período válido: cinza */
127  /* Setas de entrada e saída desenhadas na lateral da planta. */
128  .entrada { position: absolute; left: 2%; top: 35%; width: 9%; height: 32%; border-right: 2px dashed
var(--marcacao); display: flex; flex-direction: column; justify-content: space-around; text-align: center;
```

```
color: var(--secundario); font: 32px/1 var(--fonte-titulo); }
129  .entrada small { display: block; font: 11px var(--fonte-texto); }
```

- `.planta` tem `position: relative`, e as fileiras `position: absolute` com `left`, `top`, `width` e `height` em **porcentagem**. Assim o desenho mantém a proporção em qualquer largura. `aspect-ratio` mantém o formato; `min-width: 950px` impede que fique pequena demais (aí a rolagem lateral entra).
- Cada fileira é um **Grid**: `repeat(7, 1fr)` cria 7 colunas iguais. A do meio tem 8 colunas e 2 linhas (16 vagas).
- `.motos` é uma pilha de 3 linhas para as vagas de moto.
- `.vaga` é cada vaga: botão com borda, texto centralizado. `.vaga:enabled:hover` destaca ao passar o mouse; `.vaga.selecionada` pinta de laranja; `.vaga:disabled` deixa cinza (reservada ou sem período válido).

Resumo da reserva (linhas 131 a 139)

```
131  /* ---------- Resumo da reserva ---------- */
132  .vaga-escolhida { margin: 12px 0; padding: 12px; background: var(--superficie); border: 1px solid var(--
borda); }  /* caixa com o código da vaga */
133  .vaga-escolhida span, .vaga-escolhida small { display: block; color: var(--secundario); font-size: 13px;
}
134  .vaga-escolhida strong { font: 600 28px var(--fonte-titulo); }
135  /* Cada par "nome / valor" ocupa uma linha, com o nome à esquerda e o valor à direita. */
136  dl div { display: flex; justify-content: space-between; gap: 12px; padding: 8px 0; border-bottom: 1px
solid var(--borda); font-size: 14px; }
137  dt { color: var(--secundario); }
138  dd { margin: 0; font-weight: 600; text-align: right; }
139  .total dd { font-size: 20px; color: var(--destaque); }
```

- `.vaga-escolhida` é a caixa grande com o código da vaga.
- `dl div` faz cada par "nome / valor" ficar numa linha, com o nome à esquerda e o valor à direita (`justify-content: space-between`). `.total dd` destaca o total em laranja.

Layout das etapas 2 e 3 e pagamento (linhas 141 a 150)

```
141  /* ---------- Etapas 2 e 3: resumo à esquerda e formulário à direita ---------- */
142  .duas-colunas { display: grid; grid-template-columns: 344px minmax(0, 560px); gap: 32px; align-items:
start; }
143  .duas-colunas .resumo-box { padding-top: 0; border: 0; }
144  .form-box { background: var(--superficie); border: 1px solid var(--borda); padding: 24px; }
145  .form-box h2 { margin-top: 20px; }
146  .form-box h2:first-child { margin-top: 0; }
147  .pagamentos label { display: block; padding: 12px; margin-bottom: 8px; border: 1px solid var(--borda); }
148  .pagamentos label:has(:checked) { border-color: var(--destaque); background: var(--destaque-suave); }
/* :has() destaca a opção marcada, sem JavaScript */
149  .pagamentos small { display: block; margin-left: 24px; color: var(--secundario); }
150  #pix { display: block; margin-top: 12px; }  /* QR Code de demonstração */
```

- `.duas-colunas`: resumo de 344 px e formulário de até 560 px.
- `.pagamentos label:has(:checked)` usa o seletor `:has()` para destacar com borda laranja a opção de pagamento que está marcada, sem JavaScript.

Lista de reservas (linhas 152 a 160)

```
152  /* ---------- Minhas reservas ---------- */
153  .minhas { margin-top: 40px; }
154  .titulo-minhas { display: flex; justify-content: space-between; align-items: center; margin-bottom:
```

```
12px; }
155  #contador { background: var(--superficie); border: 1px solid var(--borda); padding: 2px 8px; font: 14px
var(--fonte-texto); }
156  .recibo { display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap:
12px; padding: 16px; margin-bottom: 8px; background: var(--superficie); border: 1px solid var(--borda); }
157  .recibo > div { display: flex; gap: 16px; align-items: center; }
158  .recibo strong.codigo { font: 600 28px var(--fonte-titulo); color: var(--destaque); min-width: 56px; }
/* código da vaga em laranja */
159  .recibo small { display: block; color: var(--secundario); }
160  .recibo .acoes button { margin-left: 8px; }
```

- Cada reserva (`.recibo`) é uma caixa flex: o código da vaga em laranja e grande, os dados ao lado e os botões à direita. `flex-wrap: wrap` permite quebrar de linha em telas pequenas.

Rodapé e janela modal (linhas 162 a 168)

```
162  /* ---------- Rodapé e janela de saída ---------- */
163  .rodape { padding: 20px 0 32px; border-top: 1px solid var(--borda); color: var(--secundario); font-size:
14px; }
164  dialog { max-width: 440px; width: calc(100% - 32px); padding: 24px; border: 1px solid var(--borda);
background: var(--superficie); color: var(--texto); }
165  dialog::backdrop { background: #0008; }  /* escurece o fundo atrás da janela */
166  .acoes { display: flex; gap: 8px; }
167  .acoes .principal, .acoes .botao { margin-top: 16px; }
168  #resultado-saida p { display: flex; justify-content: space-between; margin-top: 6px; }
```

- `dialog::backdrop` escurece o fundo atrás da janela modal.
- `#resultado-saida p` alinha cada linha do cálculo de saída (nome à esquerda, valor à direita).

Responsividade (linhas 170 a 176)

```
170  /* ---------- Telas menores (até 1000 px): tudo em uma coluna ---------- */
171  @media (max-width: 1000px) {
172    .miolo { width: calc(100% - 32px); }
173    .reserva { grid-template-columns: 1fr; grid-template-areas: "periodo" "planta" "resumo"; }  /* ordem:
período, planta, resumo */
174    .duas-colunas { grid-template-columns: 1fr; }
175    .etapas li { font-size: 12px; }
176  }
```

- `@media (max-width: 1000px)` vale só em telas de até 1000 px de largura. Nela, a grade vira **uma coluna** e a ordem passa a ser período, planta e resumo. As etapas 2 e 3 também empilham.

Parte 3 - script.js

> O arquivo segue a jornada do usuário: dados e constantes, planta, período e regras, etapa 1, navegação, etapa 2, etapa 3, minhas reservas, saída e tema.

Constantes de tarifa, tempo e chave (linhas 1 a 13)

```
 1  // ParkingHub - lógica do sistema de reservas (versão simples).
 2  // Ordem do arquivo: constantes -> dados -> planta -> regras -> etapas 1, 2 e 3 -> reservas -> saída -> tema.
 3
 4  /* ---------- Constantes e dados ---------- */
 5
 6  // Valores de exemplo: altere só estes números para mudar as tarifas (em reais).
 7  const TARIFAS = { hora: 8, diaria: 35, horaExtra: 5 };
 8
 9  // 1 hora em milissegundos (60 x 60 x 1000). O JavaScript trabalha com datas em milissegundos.
10  const MS_POR_HORA = 36e5;
11
12  // Nome da "gaveta" onde as reservas ficam guardadas no navegador (localStorage).
13  const CHAVE_ARMAZENAMENTO = "parkinghub.simples.v2";
```

- `const` cria uma variável que **não pode ser reatribuída**. `TARIFAS` é um **objeto** com três propriedades, lidas assim: `TARIFAS.hora` (8).
- `MS_POR_HORA = 36e5` é a notação científica de 3.600.000: o número de **milissegundos** em uma hora (60 x 60 x 1000). O JavaScript representa datas como milissegundos, o que facilita somar e comparar horários.
- `CHAVE_ARMAZENAMENTO` é o nome sob o qual as reservas são guardadas no `localStorage`.

As formas de pagamento (linhas 15 a 23)

```
15  // Formas de pagamento. A chave é igual ao "value" do botão de rádio no HTML.
16  // pagaNoLocal = true  -> a reserva fica com pagamento pendente (paga no estacionamento).
17  // pagaNoLocal = false -> o pagamento é apenas simulado (Wallet e Pix).
18  const PAGAMENTOS = {
19    cartao: { nome: "Cartão de crédito", pagaNoLocal: true, descricao: "Você paga com cartão de crédito ao chegar ao estacionamento." },
20    "google-wallet": { nome: "Google Wallet", pagaNoLocal: false, descricao: "Simulação da confirmação pela carteira digital." },
21    pix: { nome: "Pix", pagaNoLocal: false, descricao: "Veja o QR Code de demonstração e confirme para simular o pagamento." },
22    dinheiro: { nome: "Dinheiro", pagaNoLocal: true, descricao: "Você paga em dinheiro ao chegar ao estacionamento." }
23  };
```

- Um objeto cujas propriedades são as quatro formas. A chave é igual ao `value` do radio no HTML. `"google-wallet"` precisa de aspas por ter hífen.
- `pagaNoLocal: true` significa "paga no estacionamento": a reserva fica com pagamento **pendente**. `pagaNoLocal: false` (Wallet e Pix) é só simulação.
- A `descricao` é a descrição mostrada na etapa 3. Esse objeto é usado na etapa 3 e na lista de reservas.

Funções auxiliares (linhas 25 a 46)

```
25  // Atalho: $("nome") devolve o elemento da página que tem id="nome".
26  const $ = id => document.getElementById(id);
27
28  // Formata um número como dinheiro brasileiro: 16 -> "R$ 16,00".
29  const moeda = valor => valor.toLocaleString("pt-BR", { style: "currency", currency: "BRL" });
30
31  // Formata milissegundos como data e hora: "26/10/2026, 08:00".
32  const dataHora = milissegundos => new Date(milissegundos).toLocaleString("pt-BR", { dateStyle: "short",
timeStyle: "short" });
33
34  // Protege contra XSS: troca & < > " por códigos HTML, para o texto digitado nunca virar código.
35  const escaparHtml = texto => texto.replace(/[&<>"]/g, caractere => "&#" + caractere.charCodeAt() + ";");
36
37  // Devolve a modalidade marcada no formulário: "horario" ou "diaria".
38  const modalidade = () => document.querySelector('[name="modalidade"]:checked').value;
39
40  // Devolve o HTML de um ícone da biblioteca definida no index.html.
41  // nome = "moto", "acessivel", "lua" ou "sol". O ícone herda a cor do texto onde for colocado.
42  const icone = nome => `<svg class="icone-svg" aria-hidden="true"><use href="#icone-${nome}"/></svg>`;
43
44  // Converte milissegundos para o texto "AAAA-MM-DDTHH:MM" no horário local do usuário.
45  // (toISOString trabalha em UTC, por isso o fuso horário é descontado antes.)
46  const paraTextoLocal = milissegundos => new Date(milissegundos - new Date().getTimezoneOffset() *
60000).toISOString().slice(0, 16);
```

- `const $ = id => document.getElementById(id)` é uma **função de seta** (arrow function): recebe `id` e devolve o elemento da página. É um atalho: `$("nome")` em vez de `document.getElementById("nome")`.
- `moeda`: `toLocaleString("pt-BR", {style: "currency", currency: "BRL"})` formata 16 como **R$ 16,00**.
- `dataHora`: recebe milissegundos, cria uma `Date` e formata como "26/10/2026, 08:00".
- `escaparHtml` **protege contra XSS**: troca os caracteres `& < > "` pelo código HTML equivalente (`charCodeAt()` devolve o número do caractere). Assim, se alguém digitar `<b>` no nome, o texto aparece literal e não vira HTML.
- `icone(nome)` devolve o HTML de um ícone (`<svg><use href="#icone-moto">`). Os desenhos ficam num bloco `<symbol>` invisível no `index.html` e são reaproveitados em vários lugares; usam a cor do texto onde forem colocados. `modalidade()` procura o radio marcado (`:checked`) e devolve seu `value`: "horario" ou "diaria".

Estado do programa e localStorage (linhas 48 a 58)

```
48  // Lista com todas as reservas. Começa com o que estava salvo no navegador (ou vazia).
49  let reservas = [];
50  try { reservas = JSON.parse(localStorage.getItem(CHAVE_ARMAZENAMENTO) || "[]"); } catch {}
51
52  // Grava a lista de reservas no navegador. JSON.stringify transforma a lista em texto.
53  // O try/catch evita erro em navegadores que bloqueiam o armazenamento.
54  const salvar = () => { try { localStorage.setItem(CHAVE_ARMAZENAMENTO, JSON.stringify(reservas)); }
catch {} };
55
56  let vagaSelecionada = null; // vaga clicada na planta (ou null quando nenhuma está escolhida)
57  let dadosCliente = null;    // nome, telefone e placa já validados (passam da etapa 2 para a 3)
58  let reservaEmSaida = null;  // reserva aberta na janela de simulação de saída
```

- `let` cria variável que pode mudar. `reservas` é um **array** (lista) com todas as reservas.

- `try { ... } catch {}` tenta executar e, se der erro, ignora. `localStorage.getItem(CHAVE_ARMAZENAMENTO)` devolve o texto salvo, ou `null` se nunca salvou; o `|| "[]"` troca o `null` por uma lista vazia em texto. `JSON.parse` converte o texto de volta em objetos.
- `salvar` faz o caminho inverso: `JSON.stringify` transforma o array em texto e `setItem` guarda no navegador. O `try/catch` existe porque alguns navegadores bloqueiam o armazenamento.
- `vagaSelecionada`: vaga clicada na planta (ou `null`). `dadosCliente`: nome, telefone e placa já validados, que passam da etapa 2 para a 3. `reservaEmSaida`: reserva aberta no modal de saída.

Criando uma vaga (linhas 60 a 84)

```
60  /* ---------- Planta: 26 vagas de carro (V), 6 de moto (M) e 2 acessíveis (D) ---------- */
61
62  // Lista de todas as vagas criadas. Cada item é { codigo, tipo, botao }.
63  const vagas = [];
64
65  // Cria o botão de uma vaga e o coloca dentro de uma fileira da planta.
66  function criarVaga(codigo, tipo, fileira, classeExtra = "") {
67    const botao = document.createElement("button");
68    botao.type = "button"; // não envia o formulário ao clicar
69    botao.className = "vaga " + classeExtra;
70    // Código da vaga e, nas vagas de moto e acessíveis, o ícone correspondente ao lado.
71    botao.innerHTML = codigo + (tipo == "Moto" ? icone("moto") : tipo == "Acessível" ? icone("acessivel")
: "");
72    botao.title = `${codigo} · ${tipo}`; // texto que aparece ao passar o mouse
73    const vaga = { codigo, tipo, botao };
74
75    // Ao clicar: se a vaga já estava selecionada, desmarca; senão, seleciona. Depois redesenha a tela.
76    botao.onclick = () => {
77      vagaSelecionada = vagaSelecionada == vaga ? null : vaga;
78      $("aviso").hidden = true;
79      atualizar();
80    };
81
82    vagas.push(vaga);
83    fileira.append(botao);
84  }
```

- `vagas` é a lista que guarda as 34 vagas (cada uma com código, tipo e o botão).
- `criarVaga` faz um botão: `createElement("button")` cria o elemento, `className` define as classes de CSS e `innerHTML` o conteúdo: o código da vaga mais, nas vagas de moto e acessíveis, um **ícone SVG** (função `icone()`). O operador `cond ? a : b` (ternário) escolhe qual ícone usar.
- `botao.onclick` define o que acontece ao clicar: se a vaga clicada já é a selecionada, **desmarca** (`null`); senão, **seleciona**. Depois esconde o aviso e chama `atualizar()` para redesenhar.
- `const vaga = { codigo, tipo, botao }` cria um objeto (a forma curta repete o nome da variável como nome da propriedade). Ele é adicionado à lista com `push` e o botão é colocado na fileira com `pai.append(botao)`.

Repetindo vagas: motos e carros (linhas 86 a 97)

```
86  // Cria uma pilha com 3 vagas de moto (M) em sequência, começando em "primeira".
87  function motos(fileira, primeira) {
88    const pilha = document.createElement("div");
89    pilha.className = "motos";
90    fileira.append(pilha);
91    for (let numero = primeira; numero < primeira + 3; numero++) criarVaga("M" + numero, "Moto", pilha,
```

```
"moto");
 92  }
 93
 94  // Cria vagas de carro (V) do número "de" até o número "ate".
 95  function carros(fileira, de, ate) {
 96    for (let numero = de; numero <= ate; numero++) criarVaga("V" + numero, "Carro", fileira);
 97  }
```

- `motos` cria uma `div.motos` (pilha) e, com um laço `for`, cria 3 vagas de moto em sequência (`"M" + n` junta o texto M com o número).
- `carros` cria as vagas V de um número até outro.
- Os laços `for (let n = de; n <= ate; n++)` têm três partes: início, condição de continuar e passo.

Montando a planta na ordem do desenho (linhas 99 a 107)

```
 99  // Monta a planta na mesma ordem do desenho do estacionamento.
100  motos($("fileira-topo"), 1);                    // fileira de cima: M1 a M3
101  carros($("fileira-topo"), 1, 6);                //                  V1 a V6
102  criarVaga("D1", "Acessível", $("fileira-meio"), "acessivel"); // fileira do meio: D1, V7 a V13, D2, V14
a V20
103  carros($("fileira-meio"), 7, 13);
104  criarVaga("D2", "Acessível", $("fileira-meio"), "acessivel");
105  carros($("fileira-meio"), 14, 20);
106  motos($("fileira-base"), 4);                    // fileira de baixo: M4 a M6
107  carros($("fileira-base"), 21, 26);              //                   V21 a V26
```

- Cada chamada coloca vagas dentro de uma fileira do HTML, na ordem em que aparecem no desenho.
- Fileira de cima: motos M1 a M3 e V1 a V6. Do meio: D1, V7 a V13, D2, V14 a V20 (16 vagas em 8 colunas e 2 linhas). De baixo: motos M4 a M6 e V21 a V26.
- Total: 26 de carro + 6 de moto + 2 acessíveis = **34 vagas**.

periodo(): lê o período do formulário (linhas 109 a 128)

```
109  /* ---------- Período e regras de ocupação ---------- */
110
111  // Lê data e horários do formulário e calcula o período e o valor.
112  // Devolve { inicio, fim, valor } (em milissegundos e reais) ou { erro } com a mensagem do problema.
113  function periodo() {
114    const dia = $("data").value; // texto como "2026-10-26"
115    // `${dia}T${hora}` forma "2026-10-26T08:00"; o "+" na frente transforma a data em número
(milissegundos).
116    const inicio = +new Date(`${dia}T${$("entrada").value}`);
117    if (!(inicio > 0)) return { erro: "Escolha uma data e um horário de entrada válidos." }; // data
inválida vira NaN
118
119    // Diária: o fim é o início + 24 horas e o valor é fixo.
120    if (modalidade() == "diaria") return { inicio, fim: inicio + 24 * MS_POR_HORA, valor: TARIFAS.diaria
};
121
122    // Por horário: a saída precisa ser depois da entrada, no mesmo dia.
123    const fim = +new Date(`${dia}T${$("saida").value}`);
124    if (!(fim > inicio)) return { erro: "A saída precisa ser depois da entrada, no mesmo dia. Para 24
horas, escolha a diária." };
125
126    // Math.ceil arredonda as horas para cima: 2h30 cobra 3 horas.
127    return { inicio, fim, valor: Math.ceil((fim - inicio) / MS_POR_HORA) * TARIFAS.hora };
128  }
```

- Devolve um objeto com `inicio`, `fim` e `valor`, ou um objeto com `erro` (mensagem).
- `$("data").value` é um texto como "2026-10-26". O template `` `${dia}T${hora}` `` junta no formato "2026-10-26T08:00". O `new Date(...)` interpreta isso e o `+` na frente converte a data em um **número (milissegundos)**.
- Se a data for inválida, o resultado é `NaN`. A comparação `!(ini > 0)` é verdadeira para `NaN` e portanto pega esse caso.
- **Diária**: o fim é o início + 24 horas e o valor é fixo (R$ 35).
- **Por horário**: se a saída não for depois da entrada, devolve erro. Senão, o valor é `Math.ceil(horas) x 8`. `Math.ceil` arredonda **para cima**: 2h30 cobra 3 horas.

conflito(): a regra mais importante (linhas 130 a 141)

```
130 // Dois períodos se sobrepõem quando cada um começa antes de o outro terminar.
131 // Esta função diz se já existe reserva ATIVA no mesmo período para a mesma vaga ou a mesma placa.
132 //   campo = "vaga" ou "placa"  |  valor = qual vaga ou qual placa verificar
133 function conflito(periodoAtual, campo, valor) {
134   if (periodoAtual.erro) return false; // período inválido: não há o que comparar
135   return reservas.some(reserva =>
136     reserva.status == "ativa" &&
137     reserva[campo] == valor &&
138     periodoAtual.inicio < reserva.fim &&
139     periodoAtual.fim > reserva.inicio
140   );
141 }
```

- Parâmetros: `periodoAtual` (período desejado), `campo` (`"vaga"` ou `"placa"`) e `valor` (qual vaga ou placa).
- `if (periodoAtual.erro) return false`: se o período é inválido, não há conflito.
- `reservas.some(reserva => ...)` devolve `true` se **pelo menos uma** reserva satisfaz todas as condições: está ativa, `reserva[campo] == valor` (os colchetes permitem escolher a propriedade dinamicamente) e os períodos se sobrepõem.
- **Sobreposição**: `periodoAtual.inicio < reserva.fim && periodoAtual.fim > reserva.inicio`, ou seja, cada um começa antes de o outro terminar.

Reserva existente	Nova	Conflito?
08h às 10h	09h às 11h	Sim (09 < 10 e 11 > 08)
08h às 10h	10h às 12h	Não (10 não é menor que 10)
08h às 10h	06h às 07h	Não (07 não é maior que 08)

A mesma função bloqueia a vaga (planta) e a placa (etapa 2).

atualizar(): primeira parte (linhas 143 a 159)

```
143 /* ---------- Etapa 1: atualizar planta e resumo ---------- */
144
145 // Redesenha a etapa 1. É chamada toda vez que algo muda (período, vaga, reservas).
146 function atualizar() {
147   const periodoAtual = periodo();
148   const ehDiaria = modalidade() == "diaria";
149
150   $("campo-saida").hidden = ehDiaria; // na diária não existe horário de saída
```

```
151    $("erro-periodo").textContent = periodoAtual.erro || "";
152    $("erro-periodo").hidden = !periodoAtual.erro;
153
154    // Se o novo período deixou a vaga escolhida ocupada, avisa e desfaz a escolha.
155    if (vagaSelecionada && conflito(periodoAtual, "vaga", vagaSelecionada.codigo)) {
156      $("aviso").textContent = `A vaga ${vagaSelecionada.codigo} está reservada no novo período. Escolha
outra vaga.`;
157      $("aviso").hidden = false;
158      vagaSelecionada = null;
159    }
```

- É chamada **toda vez que algo muda**. Recalcula o período e redesenha a tela.
- `$("campo-saida").hidden = ehDiaria`: atribui `true` ou `false` ao atributo `hidden`, escondendo o campo de saída na diária.
- Mostra ou esconde a mensagem de erro do período.
- Se há vaga selecionada e o novo período a deixou ocupada, mostra o aviso e **desfaz a seleção** (`vagaSelecionada = null`).

atualizar(): planta, resumo e botão (linhas 161 a 187)

```
161    // Percorre as 34 vagas: bloqueia as ocupadas, destaca a escolhida e conta as livres.
162    let livres = 0;
163    for (const vaga of vagas) {
164      const ocupada = conflito(periodoAtual, "vaga", vaga.codigo);
165      vaga.botao.disabled = Boolean(periodoAtual.erro) || ocupada;
166      vaga.botao.classList.toggle("selecionada", vaga == vagaSelecionada); // liga a classe só na
escolhida
167      if (!periodoAtual.erro && !ocupada) livres++;
168    }
169    $("disponiveis").textContent = periodoAtual.erro ? "Defina o período" : `${livres} de ${vagas.length}
disponíveis`;
170
171    // Resumo da reserva ("—" quando o período é inválido).
172    $("r-vaga").textContent = vagaSelecionada ? vagaSelecionada.codigo : "Escolha na planta";
173    $("r-tipo").textContent = vagaSelecionada ? vagaSelecionada.tipo : "Carro, moto ou vaga acessível";
174    $("r-entrada").textContent = periodoAtual.erro ? "—" : dataHora(periodoAtual.inicio);
175    $("r-saida").textContent = periodoAtual.erro ? "—" : dataHora(periodoAtual.fim);
176    $("r-total").textContent = periodoAtual.erro ? "—" : moeda(periodoAtual.valor);
177
178    // Duração: minutos totais -> horas inteiras (Math.floor) e minutos que sobram (% = resto da divisão).
179    const minutos = Math.round((periodoAtual.fim - periodoAtual.inicio) / 60000);
180    $("r-duracao").textContent = periodoAtual.erro ? "—" : ehDiaria ? "24 horas" : `${Math.floor(minutos /
60)} h${minutos % 60 ? ` ${minutos % 60} min` : ""}`;
181
182    // Botão Continuar: só liberado com período válido e vaga escolhida.
183    $("continuar").disabled = Boolean(periodoAtual.erro) || !vagaSelecionada;
184    $("continuar").textContent = periodoAtual.erro
185      ? "Revise a data e os horários"
186      : vagaSelecionada ? `Continuar com a vaga ${vagaSelecionada.codigo}` : "Selecione uma vaga";
187  }
```

- `for (const vaga of vagas)` percorre as 34 vagas. Para cada uma: calcula se está `ocupada` e define `botao.disabled` (bloqueado se há erro ou está ocupada). `classList.toggle("sel", v == selecionada)` liga a classe `sel` (laranja) só na vaga escolhida. Conta também as livres.
- Escreve "33 de 34 disponíveis". `vagas.length` é o tamanho da lista.
- Preenche o resumo. `periodoAtual.erro ? "—" : valor` mostra um traço quando o período é inválido.

- Duração: `minutos` são os minutos totais; `Math.floor(minutos / 60)` são as horas inteiras; `minutos % 60` (resto da divisão) são os minutos que sobram.
- O botão "Continuar" fica desabilitado se há erro ou nenhuma vaga escolhida, e o texto muda conforme a situação.

mostrar(): troca de etapa (linhas 189 a 207)

```
189 /* ---------- Navegação entre as 3 etapas ---------- */
190
191 let etapa = 0; // etapa atual: 0 = vaga e período, 1 = seus dados, 2 = pagamento
192
193 // Mostra a etapa "numero" e esconde as outras (o atributo "hidden" esconde o elemento).
194 function mostrar(numero) {
195   etapa = numero;
196   $("tela-vaga").hidden = numero != 0;
197   $("tela-confirmar").hidden = numero == 0;
198   $("form-dados").hidden = numero != 1;
199   $("form-pagamento").hidden = numero != 2;
200   $("minhas").hidden = numero != 0;
201   $("c-cliente").hidden = numero != 2;
202   // Barra de progresso: "atual" na etapa em andamento, "feito" nas anteriores, sem classe nas próximas.
203   [...$("etapas").children].forEach((item, posicao) => {
204     item.className = posicao == numero ? "atual" : posicao < numero ? "feito" : "";
205   });
206   window.scrollTo(0, 0); // volta ao topo da página
207 }
```

- `etapa` guarda o número da etapa atual (0, 1 ou 2).
- `mostrar(n)` mostra e esconde as telas com `hidden`. Ex.: `$("tela-vaga").hidden = numero != 0` esconde a etapa 1 sempre que o número não for 0. Os formulários de dados (numero = 1) e de pagamento (numero = 2) usam a mesma ideia.
- `[...$("etapas").children].forEach(...)`: `children` são os 3 itens da barra; os `...` (spread) os copiam para um array; `forEach` percorre cada um com sua posição (`posicao`). O ternário aninhado escolhe a classe: `"atual"` se é a etapa atual, `"feito"` se vem antes, ou vazio.
- `window.scrollTo(0, 0)` volta ao topo da página.

preencherResumo() (linhas 209 a 218)

```
209 // Preenche o resumo lateral das etapas 2 e 3 com a vaga e o período escolhidos.
210 function preencherResumo() {
211   const periodoAtual = periodo();
212   $("c-vaga").textContent = vagaSelecionada.codigo;
213   $("c-tipo").textContent = vagaSelecionada.tipo;
214   $("c-mod").textContent = modalidade() == "diaria" ? "Diária de 24 h" : "Por horário";
215   $("c-entrada").textContent = dataHora(periodoAtual.inicio);
216   $("c-saida").textContent = dataHora(periodoAtual.fim);
217   $("c-total").textContent = moeda(periodoAtual.valor);
218 }
```

- Copia a vaga, o tipo, a modalidade, as datas e o total para o resumo lateral das etapas 2 e 3.

Botão Continuar (etapa 1 para 2) (linhas 220 a 238)

```
220  // Botão "Continuar" da etapa 1: confere tudo e vai para a etapa 2.
221  $("tela-vaga").onsubmit = evento => {
222    evento.preventDefault(); // impede o comportamento padrão do formulário (recarregar a página)
223    const periodoAtual = periodo();
224    if (periodoAtual.erro || !vagaSelecionada || conflito(periodoAtual, "vaga", vagaSelecionada.codigo))
return atualizar();
225
226    const acessivel = vagaSelecionada.tipo == "Acessível";
227    $("ajuda-tipo").textContent = acessivel
228      ? `Você escolheu a vaga ${vagaSelecionada.codigo}, acessível para carro. Anexe a credencial abaixo.`
229      : `Você escolheu a vaga ${vagaSelecionada.codigo}, destinada a ${vagaSelecionada.tipo == "Moto" ?
"motos" : "carros"}.`;
230    $("campo-cred").hidden = !acessivel; // só vaga acessível pede credencial
231    preencherResumo();
232    mostrar(1);
233  };
234
235  $("voltar").onclick = () => mostrar(etapa - 1);
236
237  // Qualquer mudança nos campos da etapa 1 (data, hora, modalidade) redesenha a tela.
238  $("tela-vaga").oninput = () => { $("aviso").hidden = true; atualizar(); };
```

- `onsubmit` roda quando o formulário é enviado. `evento.preventDefault()` **impede o comportamento padrão** (recarregar a página).
- Se o período é inválido, não há vaga ou a vaga ficou ocupada, chama `atualizar()` e para (`return`).
- Se a vaga é acessível, mostra o campo da credencial e ajusta o texto de ajuda. Depois preenche o resumo e vai para a etapa 2 (`mostrar(1)`).
- `$("voltar").onclick` volta uma etapa. `$("tela-vaga").oninput` reage a qualquer digitação ou mudança de campo no formulário da etapa 1, redesenhando tudo.

Normalização de placa e telefone (linhas 240 a 260)

```
240  /* ---------- Etapa 2: dados do cliente ---------- */
241
242  // Placa em maiúsculas e sem espaços ou hífens: "abc-1d23" -> "ABC1D23".
243  // /[\s-]/g é uma expressão regular: espaço ou hífen, em todas as ocorrências (g).
244  const normalizarPlaca = texto => texto.toUpperCase().replace(/[\s-]/g, "");
245
246  // Telefone só com dígitos, sem o código do país: "+55 (11) 99999-8888" -> "11999998888".
247  function normalizarTelefone(texto) {
248    const digitos = texto.replace(/\D/g, ""); // \D = qualquer caractere que não seja dígito
249    return /^55\d{10,11}$/.test(digitos) ? digitos.slice(2) : digitos; // se começa com 55 + 10 ou 11
dígitos, remove o 55
250  }
251
252  // Formata como "(11) 99999-8888" (10 ou 11 dígitos). Outros textos ficam como estão.
253  const formatarTelefone = telefone =>
254    /^\d{10,11}$/.test(telefone) ? `(${telefone.slice(0, 2)}) ${telefone.slice(2,
-4)}-${telefone.slice(-4)}` : telefone;
255
256  // Enquanto digita no telefone, apaga letras e símbolos não permitidos.
257  $("telefone").oninput = () => { $("telefone").value = $("telefone").value.replace(/[^\d+()\s-]/g, "");
};
258  // Ao sair do campo, formata o número. Na placa, converte para maiúsculas.
259  $("telefone").onblur = () => { $("telefone").value =
```

```
formatarTelefone(normalizarTelefone($("telefone").value)); };
260  $("placa").onblur = () => { $("placa").value = normalizarPlaca($("placa").value); };
```

- **Expressão regular (regex)** é um padrão entre barras. `/[\s-]/g` procura espaços ou hífens; o `g` (global) troca todos. `normalizarPlaca` deixa em maiúsculas e sem espaço/hífen: "abc-1d23" vira "ABC1D23".
- `normalizarTelefone`: `\D` é "qualquer coisa que não seja dígito", então `replace(/\D/g, "")` mantém só números. Se o resultado tem 55 + 10 ou 11 dígitos (código do Brasil), remove o 55 com `slice(2)`.
- `formatarTelefone`: se tem 10 ou 11 dígitos, monta "(DD) XXXXX-XXXX" com `slice`. `slice(2, -4)` pega do terceiro dígito até os 4 últimos; `slice(-4)` pega os 4 últimos.
- `oninput` do telefone apaga letras enquanto a pessoa digita; `onblur` (ao sair do campo) formata o número. O da placa converte para maiúsculas.

erroDados() (linhas 262 a 267)

```
262  // Mostra a mensagem de erro da etapa 2 e põe o cursor no campo com problema.
263  function erroDados(mensagem, campo) {
264    $("erro-dados").textContent = mensagem;
265    $("erro-dados").hidden = false;
266    campo.focus();
267  }
```

- Mostra a mensagem de erro e coloca o cursor (`focus()`) no campo com problema.

Validação dos dados (etapa 2) (linhas 269 a 310)

```
269  // Botão "Ir para pagamento": valida cada campo. Cada "return erroDados(...)" mostra o erro e
interrompe.
270  $("form-dados").onsubmit = evento => {
271    evento.preventDefault();
272    $("erro-dados").hidden = true;
273    const periodoAtual = periodo();
274    const nome = $("nome").value.trim().replace(/\s+/g, " "); // remove espaços sobrando
275    const telefone = normalizarTelefone($("telefone").value);
276    const placa = normalizarPlaca($("placa").value);
277
278    // Nome: pelo menos 2 palavras; só letras (\p{L} inclui acentos), espaço, ponto, apóstrofo e hífen.
279    if (nome.split(" ").length < 2 || !/^[\p{L} .'’-]+$/u.test(nome))
280      return erroDados("Informe seu nome completo, com nome e sobrenome.", $("nome"));
281
282    // Telefone: DDD (2 dígitos) + fixo (começa em 2 a 9 e tem mais 7 dígitos) OU celular (9 e mais 8
dígitos).
283    if (!/^[1-9]{2}(?:[2-9]\d{7}|9\d{8})$/.test(telefone))
284      return erroDados("Informe um telefone com DDD: 10 dígitos para fixo ou 11 para celular.",
$("telefone"));
285
286    // Placa: 3 letras + 1 número + 1 letra ou número + 2 números (aceita o padrão antigo e o Mercosul).
287    if (!/^[A-Z]{3}[0-9][A-Z0-9][0-9]{2}$/.test(placa))
288      return erroDados("Informe uma placa válida, como ABC-1234 ou ABC1D23.", $("placa"));
289
290    // A mesma placa não pode ter duas reservas ativas no mesmo período.
291    if (conflito(periodoAtual, "placa", placa))
292      return erroDados(`A placa ${placa} já tem uma reserva ativa neste período. Use outro horário.`,
$("placa"));
293
294    // Vaga acessível: exige credencial em JPG, PNG, WebP ou PDF, com conteúdo e até 5 MB.
295    const acessivel = vagaSelecionada.tipo == "Acessível";
296    if (acessivel) {
297      const arquivo = $("credencial").files[0];
```

```
298      const tiposPermitidos = ["image/jpeg", "image/png", "image/webp", "application/pdf"];
299      if (!arquivo || !tiposPermitidos.includes(arquivo.type) || !arquivo.size || arquivo.size > 5 * 1024
* 1024)
300        return erroDados("Anexe a credencial em JPG, PNG, WebP ou PDF, com até 5 MB.", $("credencial"));
301    }
302
303    // Tudo certo: guarda os dados (o arquivo em si NÃO é guardado, só a indicação de que foi anexado).
304    dadosCliente = { nome, telefone, placa, credencialAnexada: acessivel };
305    $("nome").value = nome;
306    $("telefone").value = formatarTelefone(telefone);
307    $("placa").value = placa;
308    $("c-nome").textContent = `${nome} · ${placa}`;
309    mostrar(2);
310  };
```

- Lê e limpa os valores: `trim()` remove espaços das pontas e `replace(/\s+/g, " ")` troca espaços repetidos por um só.
- **Nome**: `split(" ")` separa as palavras; precisa de pelo menos 2. O regex `/^[\p{L} .'’-]+$/u` aceita só letras (`\p{L}` inclui acentos), espaço, ponto, apóstrofo e hífen. `^` e `$` exigem que o texto inteiro siga o padrão.
- **Telefone**: `/^[1-9]{2}(?:[2-9]\d{7}|9\d{8})$/` : DDD de 2 dígitos (1 a 9) seguido de fixo (começa em 2-9 e tem mais 7 dígitos) **ou** celular (9 e mais 8 dígitos).
- **Placa**: `/^[A-Z]{3}[0-9][A-Z0-9][0-9]{2}$/` : 3 letras, 1 número, 1 letra ou número e 2 números. Cobre ABC1234 e ABC1D23.
- **Placa em uso**: `conflito(p, "placa", placa)` reaproveita a regra de sobreposição.
- **Credencial** (só vaga acessível): `files[0]` é o arquivo escolhido. Confere o tipo (`tiposPermitidos.includes`), se tem conteúdo (`arquivo.size`) e o limite de 5 MB (`5 * 1024 * 1024` bytes).
- Cada `return erroDados(...)` mostra o erro e interrompe a função. Se tudo passou, guarda os dados em `dadosCliente` , formata os campos e vai para a etapa 3.

Escolha da forma de pagamento (linhas 312 a 327)

```
312  /* ---------- Etapa 3: pagamento ---------- */
313
314  // Devolve a forma de pagamento marcada (ex.: "pix") ou undefined se nenhuma estiver marcada.
315  // O "?." evita erro quando não há nada marcado.
316  const formaDePagamento = () => document.querySelector('[name="pagamento"]:checked')?.value;
317
318  // Ao escolher uma forma de pagamento: mostra a descrição, o QR Code (só no Pix) e ajusta o botão.
319  $("form-pagamento").onchange = () => {
320    const forma = PAGAMENTOS[formaDePagamento()];
321    $("pg-titulo").textContent = forma.nome;
322    $("pg-texto").textContent = forma.descricao;
323    $("pix").hidden = formaDePagamento() != "pix";
324    $("c-rotulo").textContent = forma.pagaNoLocal ? "Total a pagar no local" : "Total da reserva";
325    $("pagar").textContent = forma.pagaNoLocal ? "Reservar e pagar no local" : "Simular pagamento e
reservar";
326    $("pagar").disabled = false;
327  };
```

- `formaDePagamento()` devolve o `value` do radio marcado. O `?.` (encadeamento opcional) evita erro quando nada está marcado: devolve `undefined` .
- `onchange` roda quando o usuário escolhe uma opção: mostra o nome e a descrição (vindos de `PAGAMENTOS`), exibe o QR Code só no Pix, ajusta o rótulo do total ("Total a pagar no local" ou "Total da reserva"), troca o texto do botão e o habilita.

Confirmando e criando a reserva (linhas 329 a 374)

```
329  // Botão final: confere a vaga de novo, CRIA a reserva, salva e volta para a etapa 1.
330  $("form-pagamento").onsubmit = evento => {
331    evento.preventDefault();
332    const periodoAtual = periodo();
333    if (!formaDePagamento()) return;
334
335    // Confere de novo: outra aba do navegador poderia ter reservado a vaga enquanto o formulário estava
aberto.
336    if (periodoAtual.erro || conflito(periodoAtual, "vaga", vagaSelecionada.codigo)) {
337      mostrar(0);
338      return atualizar();
339    }
340
341    const reserva = {
342      id: Date.now(), // horário atual em milissegundos: serve como identificador único
343      vaga: vagaSelecionada.codigo,
344      tipo: vagaSelecionada.tipo,
345      nome: dadosCliente.nome,
346      telefone: dadosCliente.telefone,
347      placa: dadosCliente.placa,
348      credencialAnexada: dadosCliente.credencialAnexada,
349      pagamento: formaDePagamento(),
350      pagamentoSimulado: !PAGAMENTOS[formaDePagamento()].pagaNoLocal, // true = Pix/Wallet; false =
pendente (cartão/dinheiro)
351      modalidade: modalidade(),
352      inicio: periodoAtual.inicio,
353      fim: periodoAtual.fim,
354      valor: periodoAtual.valor,
355      adicional: 0,    // taxa de atraso, calculada só na saída
356      status: "ativa"  // pode virar "cancelada" ou "encerrada"
357    };
358    reservas.push(reserva); // acrescenta ao final da lista
359    salvar();
360
361    // Mensagem de sucesso, limpeza dos formulários e volta para a etapa 1.
362    $("aviso").textContent = `Reserva da vaga ${reserva.vaga} confirmada para ${reserva.nome}, placa
${reserva.placa}. Total: ${moeda(reserva.valor)}.`;
363    $("aviso").hidden = false;
364    vagaSelecionada = null;
365    $("form-dados").reset();
366    $("form-pagamento").reset();
367    $("pix").hidden = true;
368    $("pagar").disabled = true;
369    $("pg-titulo").textContent = "Escolha uma opção acima";
370    mostrar(0);
371    atualizar();
372    renderizar();
373    $("minhas").scrollIntoView();
374  };
```

- Antes de criar, **confere de novo** se a vaga continua livre (outra aba poderia ter reservado). Se não estiver, volta à etapa 1.
- Monta o objeto `reserva` com todos os dados. `id: Date.now()` usa o horário atual em milissegundos como identificador único. `pagamentoSimulado: !PAGAMENTOS[formaDePagamento()].pagaNoLocal` é `true` para Pix/Wallet (simulados) e `false` para cartão/dinheiro (pendentes). `adicional` começa em 0 e `status` em "ativa".
- `reservas.push(reserva)` acrescenta ao array e `salvar()` grava no navegador.

- Em seguida: mostra a mensagem de sucesso, limpa a seleção e os formulários (`reset()`), volta à etapa 1, redesenha a planta e a lista (`atualizar()` e `renderizar()`) e rola até "Minhas reservas".

renderizar(): desenha a lista (linhas 376 a 404)

```
376  /* ---------- Minhas reservas ---------- */
377
378  // Desenha a lista de reservas (a mais nova primeiro).
379  function renderizar() {
380    $("contador").textContent = reservas.filter(reserva => reserva.status == "ativa").length; // conta só
as ativas
381    $("limpar").disabled = !reservas.length;
382
383    // [...reservas].reverse() copia a lista e inverte a ordem, sem alterar a original.
384    // .map transforma cada reserva em um texto HTML; .join("") junta todos os textos em um só.
385    $("lista").innerHTML = [...reservas].reverse().map(reserva => {
386      const pagamento = PAGAMENTOS[reserva.pagamento].nome +
387        (reserva.status == "cancelada" ? "" : reserva.pagamentoSimulado ? " (simulado)" : " (pendente,
paga no local)");
388
389      // Reserva ativa mostra os botões; as demais mostram o status. Os botões guardam o id da reserva
390      // em atributos data-... para sabermos qual deles foi clicado.
391      const lado = reserva.status == "ativa"
392        ? `<span class="acoes"><button class="botao" data-saida="${reserva.id}">Simular saída</button>
<button class="botao" data-cancelar="${reserva.id}">Cancelar reserva</button></span>`
393        : `<span><strong>${reserva.status == "cancelada" ? "Cancelada" : "Encerrada"}</strong>` +
394          (reserva.status == "encerrada" ? `<small>Saída: ${dataHora(reserva.saidaReal)}. Adicional:
${moeda(reserva.adicional)}. Total: ${moeda(reserva.valor + reserva.adicional)}</small>` : "") + `</span>`;
395
396      // O nome digitado passa por escaparHtml() por segurança.
397      return `<article class="recibo">
398        <div><strong class="codigo">${reserva.vaga}</strong><div>
399          <p>${escaparHtml(reserva.nome)} · ${formatarTelefone(reserva.telefone)}</p>
400          <p>${dataHora(reserva.inicio)} → ${dataHora(reserva.fim)}</p>
401          <small>${reserva.tipo} · ${reserva.placa} · ${reserva.modalidade == "diaria" ? "Diária de 24 h"
: "Por horário"} · ${moeda(reserva.valor)} · ${pagamento}${reserva.credencialAnexada ? " · Credencial
anexada" : ""}</small>
402        </div></div>${lado}</article>`;
403    }).join("") || "<p class='nota'>Nenhuma reserva por enquanto. Selecione uma vaga para começar.</p>";
404  }
```

- Atualiza o contador (só reservas ativas, com `filter`) e desabilita "Limpar" se não há reservas.
- `[...reservas].reverse()` copia o array e inverte, para a reserva mais nova aparecer primeiro (sem alterar o array original).
- `.map(reserva => `...`)` transforma **cada reserva em um texto HTML**. O `join("")` junta todos em um só texto, que vai para `innerHTML` . O `|| "<p>...</p>"` mostra a mensagem de "nenhuma reserva" se o resultado for vazio.
- O nome do cliente passa por `escaparHtml()` (segurança). Os botões levam `data-cancelar="id"` ou `data-saida="id"` , atributos que guardam o id da reserva.
- Reserva **ativa** mostra os dois botões; **cancelada** ou **encerrada** mostra o status (e, se encerrada, saída, adicional e total).
- O texto do pagamento diz se foi "simulado" ou "pendente, paga no local".

Cancelar, abrir saída e limpar (linhas 406 a 438)

```
406 // Um único "ouvinte" de clique para a lista inteira (delegação de eventos):
407 // descobre qual botão foi clicado olhando os atributos data-cancelar e data-saida.
408 $("lista").onclick = evento => {
409   const botao = evento.target.closest("button");
410   if (!botao) return;
411
412   if (botao.dataset.cancelar) {
413     // Cancelar: pede confirmação, muda o status e a vaga volta a ficar livre.
414     const reserva = reservas.find(item => item.id == botao.dataset.cancelar);
415     if (!confirm(`Cancelar a reserva da vaga ${reserva.vaga}? Ela ficará disponível novamente.`))
return;
416     reserva.status = "cancelada";
417     salvar();
418     atualizar();
419     renderizar();
420   } else if (botao.dataset.saida) {
421     // Simular saída: abre a janela já preenchida com o fim contratado.
422     reservaEmSaida = reservas.find(item => item.id == botao.dataset.saida);
423     $("titulo-saida").textContent = `Simular saída da vaga ${reservaEmSaida.vaga}`;
424     $("texto-saida").textContent = `Período contratado até ${dataHora(reservaEmSaida.fim)}. Altere a
saída para testar a taxa extra.`;
425     $("saida-real").value = paraTextoLocal(reservaEmSaida.fim);
426     calcularSaida();
427     $("modal-saida").showModal();
428   }
429 };
430
431 // Botão "Limpar reservas": apaga todas, depois de confirmar.
432 $("limpar").onclick = () => {
433   if (!confirm("Apagar todas as reservas salvas neste navegador? Não é possível desfazer.")) return;
434   reservas = [];
435   salvar();
436   atualizar();
437   renderizar();
438 };
```

- **Delegação de eventos**: em vez de um `onclick` em cada botão (que seriam recriados a cada desenho), há um só na lista. `evento.target.closest("button")` descobre qual botão foi clicado, e `botao.dataset.cancelar` lê o `data-cancelar`.
- `reservas.find(item => item.id == ...)` localiza a reserva pelo id.
- **Cancelar**: `confirm()` abre uma caixa de sim/não; se confirmado, muda o status para "cancelada", salva e redesenha. A vaga volta a ficar livre porque `conflito()` só considera reservas "ativa".
- **Simular saída**: guarda a reserva em `reservaEmSaida`, escreve os textos do modal e preenche o campo de data com o fim contratado. A conta `fim - getTimezoneOffset() * 60000` corrige o fuso horário, porque `toISOString()` trabalha em UTC. `slice(0, 16)` mantém o formato "AAAA-MM-DDTHH:MM". Por fim `showModal()` abre a janela.
- **Limpar reservas**: confirma, esvazia o array e salva.

calcularSaida(): taxa de atraso (linhas 440 a 466)

```
440 /* ---------- Saída: cada hora de atraso (iniciada) custa a taxa extra ---------- */
441
442 // Calcula a taxa de atraso para o horário de saída digitado na janela.
443 // Devolve { saidaReal, adicional } ou null se o horário for inválido.
444 function calcularSaida() {
```

```
445    const saidaReal = +new Date($("saida-real").value);
446    const invalida = !(saidaReal >= reservaEmSaida.inicio); // saída antes da entrada (ou data inválida)
447    $("erro-saida").textContent = invalida ? "Informe uma saída igual ou posterior à entrada da reserva."
: "";
448    $("erro-saida").hidden = !invalida;
449    $("resultado-saida").innerHTML = "";
450    if (invalida) return null;
451
452    // Minutos depois do fim contratado. Math.max(0, ...) impede valor negativo se sair antes do fim.
453    const minutosExcedidos = Math.max(0, Math.ceil((saidaReal - reservaEmSaida.fim) / 60000));
454    // Horas iniciadas x taxa extra. Exemplo: 80 min -> 2 horas -> R$ 10.
455    const adicional = Math.ceil(minutosExcedidos / 60) * TARIFAS.horaExtra;
456
457    // Cada par [nome, valor] vira uma linha do resultado. ([nome, valor]) => ... separa o par em duas
variáveis.
458    $("resultado-saida").innerHTML = [
459      ["Reserva", moeda(reservaEmSaida.valor)],
460      ["Tempo excedido", minutosExcedidos ? `${Math.floor(minutosExcedidos / 60)} h ${minutosExcedidos %
60} min` : "Sem excedente"],
461      ["Taxa adicional", moeda(adicional)],
462      ["Total", moeda(reservaEmSaida.valor + adicional)]
463    ].map(([nome, valor]) => `<p><span>${nome}</span><b>${valor}</b></p>`).join("");
464
465    return { saidaReal, adicional };
466  }
```

- Lê a data de saída. Se for anterior à entrada da reserva (ou inválida), mostra erro e devolve `null`.
- `minutosExcedidos`: diferença entre a saída real e o fim contratado, em minutos (dividir por 60000), arredondada para cima. `Math.max(0, ...)` garante que sair **antes** do fim não vire número negativo.
- `adicional`: `Math.ceil(minutosExcedidos / 60)` são as horas iniciadas, multiplicadas por R$ 5. Ex.: 80 min vira 2 horas e R$ 10.
- As linhas do resultado vêm de uma lista de pares `[nome, valor]`. O `([a, b]) =>` é a **desestruturação**: pega os dois itens do par de uma vez.
- Devolve `{ saidaReal, adicional }` para quem chamou usar.

Fechar modal e encerrar a reserva (linhas 468 a 481)

```
468  // A cada mudança na data, recalcula o resultado na hora.
469  $("saida-real").oninput = calcularSaida;
470  $("fechar-saida").onclick = () => $("modal-saida").close();
471
472  // Botão "Encerrar reserva". O formulário tem method="dialog", então a janela fecha sozinha ao enviar.
473  $("form-saida").onsubmit = evento => {
474    const calculo = calcularSaida();
475    if (!calculo) return evento.preventDefault(); // saída inválida: mantém a janela aberta
476    // Object.assign altera o próprio objeto da reserva, que já está dentro da lista "reservas".
477    Object.assign(reservaEmSaida, { status: "encerrada", saidaReal: calculo.saidaReal, adicional:
calculo.adicional });
478    salvar();
479    atualizar();
480    renderizar();
481  };
```

- Cada alteração da data recalcula o resultado na hora (`oninput`).
- No envio: se a saída é inválida, `evento.preventDefault()` **impede o fechamento** da janela. Se é válida, `Object.assign(reservaEmSaida, {...})` altera o próprio objeto da reserva (status "encerrada", horário real

e adicional). Como esse objeto é o mesmo que está em `reservas`, a lista é atualizada. Depois salva e redesenha. O `method="dialog"` do formulário fecha a janela sozinho.

Tema, data inicial e arranque do programa (linhas 483 a 499)

```
483  /* ---------- Tema claro/escuro e início do programa ---------- */
484
485  // Liga ou desliga o tema escuro: escreve data-tema no elemento raiz (o CSS troca as cores por isso) e
troca o ícone.
486  function aplicarTema(escuro) {
487    document.documentElement.dataset.tema = escuro ? "escuro" : "claro";
488    $("tema").innerHTML = icone(escuro ? "sol" : "lua"); // no tema escuro mostra o sol (para voltar ao
claro) e vice-versa
489  }
490  $("tema").onclick = () => aplicarTema(document.documentElement.dataset.tema != "escuro");
491  // Começa no mesmo tema que o sistema operacional do usuário usa.
492  aplicarTema(matchMedia("(prefers-color-scheme: dark)").matches);
493
494  // Início: data de hoje já preenchida (assim o período é válido e a planta não aparece bloqueada).
495  $("data").value = paraTextoLocal(Date.now()).slice(0, 10);
496  $("tarifas").textContent = `${moeda(TARIFAS.horaExtra)} por hora excedente iniciada. Valores de exemplo:
${moeda(TARIFAS.hora)} por hora ou ${moeda(TARIFAS.diaria)} pela diária.`;
497  mostrar(0);     // começa na etapa 1
498  atualizar();    // desenha a planta e o resumo
499  renderizar();   // mostra as reservas que já estavam salvas
```

- `aplicarTema(escuro)` escreve `data-tema` no elemento raiz (que ativa as variáveis do CSS) e troca o ícone do botão (sol ou lua) com `icone()`.
- `matchMedia("(prefers-color-scheme: dark)").matches` pergunta ao sistema operacional se ele usa tema escuro: o site já abre combinando com o computador.
- A data de entrada começa em **hoje** (mesma correção de fuso do trecho anterior). Isso garante um período válido desde o início, e a planta não aparece bloqueada.
- As últimas linhas executam tudo pela primeira vez: `mostrar(0)` (etapa 1), `atualizar()` (planta e resumo) e `renderizar()` (reservas salvas anteriormente).

Resumo para fechar a apresentação

- **Fluxo:** escolher período, escolher vaga, preencher dados, pagar, ver em "Minhas reservas".
- **Regras principais:** `periodo()` calcula o valor; `conflito()` impede reservas sobrepostas de vaga e de placa; `calcularSaida()` cobra R$ 5 por hora de atraso.
- **Dados:** tudo em um array `reservas` salvo em `localStorage` como JSON. Sem servidor.
- **Interface:** o JS só liga e desliga classes e atributos; o CSS decide o visual (inclusive o tema escuro, por variáveis).
- **Limites:** é um protótipo: pagamento simulado, dados só no navegador de quem usou, credencial apenas conferida (não guardada).